

📄 ТЗ: MVP боя мехов (Blueprint-only, XR + Screen + Server)

🎯 ЦЕЛЬ

Собрать сетевой VR-прототип, где:

- прицел и цель считаются **в экранном пространстве**
 - попадание считается **на сервере**
 - есть **видеопоток от лица игроков**
 - архитектура позволяет заменить AI на реального игрока без переделки
-

❓ АРХИТЕКТУРА

Разделение:

1. Player (ввод, камера)
 2. Aiming (прицел)
 3. TargetProvider (источник цели)
 4. Combat (попадание)
 5. Networking (RPC, репликация)
 6. Video (SceneCapture)
-

📁 СПИСОК BLUEPRINT

Player

- BP_VRPlayerPawn
 - BP_PlayerController
 - BP_PlayerState
-

Gameplay

- BP_AimingComponent
 - BP_CombatComponent
 - BP_TargetProvider_Base
 - BP_TargetProvider_AI
 - BP_TargetActor_AI
-

Networking

- BP_GameMode
 - BP_GameState
-

Video

- BP_VideoCaptureComponent
 - BP_VideoScreenActor
-

🎮 PLAYER

BP_VRPlayerPawn

Компоненты:

- Camera (VR Camera)
- MotionController (L/R)
- BP_AimingComponent

- BP_CombatComponent
- BP_VideoCaptureComponent

Переменные:

- PlayerID
- bIsLocalPlayer

BeginPlay:

if IsLocallyControlled:

bIsLocalPlayer = true

BP_PlayerController

Функции:

FireInput()

Логика:

OnTriggerPressed →

Pawn.CombatComponent → RequestFire()

BP_PlayerState

Переменные (Replicated):

- HP
- PlayerName

AIMING

BP_AimingComponent

Переменные:

- AimScreenPos (Vector2D)
- ScreenCenter
- UseControllerOffset (bool)
- AimOffsetScale = 400
- MaxOffsetClamp = 300

Tick (Client)

1. Центр экрана

GetViewportSize → /2 → ScreenCenter

2. Базовый прицел

AimScreenPos = ScreenCenter

3. Offset (если включен)

Get Camera Forward

Get Controller Forward

DeltaRot = ControllerRot - CameraRot

OffsetX = Yaw * Scale

OffsetY = Pitch * Scale * -1

Clamp → MaxOffsetClamp

AimScreenPos += Offset

TARGET

BP_TargetProvider_Base

Функции:

GetTargetScreenPosition(PlayerController)
IsTargetVisible()
GetTargetActor()

BP_TargetProvider_AI

Переменные:

- TargetActor (BP_TargetActor_AI)
-

Логика (Server):

GetActorLocation(TargetActor)
ProjectWorldToScreen(PlayerController)
return Vector2D

BP_TargetActor_AI

Компоненты:

- StaticMesh
-

Переменные:

- HP (Replicated)
-

Движение:

Tick:
 $X += \sin(\text{time}) * \text{speed}$

COMBAT

BP_CombatComponent

Переменные:

- AimRadius = 120
 - LockTime = 0.3
 - LockProgress
 - TargetProvider
-

ФУНКЦИИ

RequestFire (Client)

Get AimScreenPos
Call Server_Fire(AimScreenPos)

Server_Fire (Run on Server)

Validate AimScreenPos

TargetPos = TargetProvider.GetTargetScreenPosition(PlayerController)

Distance = |Aim - Target|

if Distance < AimRadius:
 ApplyDamage

ApplyDamage

TargetActor.HP -= Damage

if HP <= 0:
 Destroy / Respawn

NETWORKING

BP_GameMode (Server)

BeginPlay:

Spawn AI Target
Assign TargetProvider

BP_GameState (Replicated)

Переменные:

- Players Array
 - MatchState
-

VIDEO SYSTEM

BP_VideoCaptureComponent

Компоненты:

- SceneCaptureComponent2D
 - TextureRenderTarget2D
-

Настройки:

- Capture Source: Final Color
 - Resolution: 512–1024
-

BeginPlay:

Attach SceneCapture to Camera
Assign RenderTarget

BP_VideoScreenActor

Компоненты:

- Plane Mesh
-

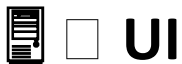
Материал:

Texture = RenderTarget

ЛОГИКА

Каждый игрок:
имеет свой RenderTarget
Экран:

показывает нужного игрока



Создать Widget:

- Crosshair (центр)
 - Target dot
 - Lock indicator
 - HP bar
-

DEBUG

Выводить:

AimScreenPos

TargetScreenPos

Distance

Ping

РІСО ОПТИМИЗАЦИЯ

- не более 1 SceneCapture на игрока
 - ограничить FPS
 - простой UI
-

ПОРЯДОК СБОРКИ

Этап 1

Player + Camera

Этап 2

Aiming (центр)

Этап 3

AI Target

Этап 4

Combat (локально)

Этап 5

Server RPC

Этап 6

Video Capture

Этап 7

Offset

РАСШИРЕНИЕ

Без переделки:

AI → Player TargetProvider

НЕ ДЕЛАТЬ

- OpenCV
 - QR
 - сложную физику
 - peer-to-peer
-

КРИТЕРИЙ УСПЕХА

- игрок понимает стрельбу
 - попадания предсказуемы
 - работает в сети
 - видео отображается
 - AI легко заменяется игроком
-

СУТЬ

- ☞ Всё в экранном пространстве
- ☞ Сервер решает попадание
- ☞ Источник цели заменяемый